

1415. The k-th Lexicographical String of All Happy Strings of Length n

Solved 

Medium

 Topics

 Companies

 Hint

A **happy string** is a string that:

- consists only of letters of the set `['a', 'b', 'c']`.
- `s[i] != s[i + 1]` for all values of `i` from `1` to `s.length - 1` (string is 1-indexed).

For example, strings **"abc"**, **"ac"**, **"b"** and **"abcbabcbcb"** are all happy strings and strings **"aa"**, **"baa"** and **"ababbc"** are not happy strings.

Given two integers `n` and `k`, consider a list of all happy strings of length `n` sorted in lexicographical order.

Return *the kth string* of this list or return an **empty string** if there are less than `k` happy strings of length `n`.

Example 1:

Input: n = 1, k = 3

Output: "c"

Explanation: The list ["a", "b", "c"] contains all happy strings of length 1. The third string is "c".

Example 2:

Input: n = 1, k = 4

Output: ""

Explanation: There are only 3 happy strings of length 1.

Example 3:

Input: n = 3, k = 9

Output: "cab"

Explanation: There are 12 different happy string of length 3 ["aba", "abc", "aca", "acb", "bab", "bac", "bca", "bcb", "cab", "cac", "cba", "cbc"]. You will find the 9th string = "cab"

Constraints:

- $1 \leq n \leq 10$
- $1 \leq k \leq 100$

Python:

#translated using AI

class Solution:

def __init__(self):

self.count = 0

def getHappyString(self, n: int, k: int) -> str:

size = 3 * (2 ** (n - 1))

if k > size:

return ""

return self.helper(n, k, "")

def helper(self, n, k, s):

```

    if len(s) == n:
        self.count += 1

    if self.count == k:
        return s

    return ""

    if len(s) == 0 or s[-1] != 'a':
        res = self.helper(n, k, s + 'a')
        if res != "":
            return res

    if len(s) == 0 or s[-1] != 'b':
        res = self.helper(n, k, s + 'b')
        if res != "":
            return res

    if len(s) == 0 or s[-1] != 'c':
        res = self.helper(n, k, s + 'c')
        if res != "":
            return res

    return ""

```

JavaScript:

```

var getHappyString = function(n, k) {
    let happy = [];
    let chars = ['a', 'b', 'c'];

    function generateHappyString(path) {
        if (path.length === n) {
            happy.push(path);
            return;
        }
        for (let c of chars) {
            if (!path || path[path.length - 1] !== c) {
                generateHappyString(path + c);
            }
        }
    }

    generateHappyString("");

```

```
    return k > happy.length ? "" : happy[k - 1];  
};
```

Java:

```
class Solution {  
    int count = 0;  
  
    public String getHappyString(int n, int k) {  
        int size = 3 * (int)Math.pow(2, n - 1);  
  
        if (k > size) return "";  
  
        StringBuilder sb = new StringBuilder();  
        return helper(n, k, sb);  
    }  
  
    public String helper(int n, int k, StringBuilder sb) {  
  
        if (sb.length() == n) {  
            count++;  
  
            if (count == k) {  
                return sb.toString();  
            }  
  
            return "";  
        }  
  
        if (sb.length() == 0 || sb.charAt(sb.length() - 1) != 'a') {  
            sb.append('a');  
            String s = helper(n, k, sb);  
            if (!s.equals("")) return s;  
            sb.deleteCharAt(sb.length() - 1);  
        }  
  
        if (sb.length() == 0 || sb.charAt(sb.length() - 1) != 'b') {  
            sb.append('b');  
            String s = helper(n, k, sb);  
            if (!s.equals("")) return s;  
            sb.deleteCharAt(sb.length() - 1);  
        }  
  
        if (sb.length() == 0 || sb.charAt(sb.length() - 1) != 'c') {  
            sb.append('c');
```

```
        String s = helper(n, k, sb);  
        if (!s.equals("")) return s;  
        sb.deleteCharAt(sb.length() - 1);  
    }  
  
    return "";  
}  
}
```